Mesos - Mini report finale

Benedetto Oliva, Giorgio Ortona, Alessandro Mazzeo, Camillo Nicoletti

FUNZIONALITA IMPLEMENTATE

Regole complete per 2-5 giocatori: lobby, ready check, ere, offer tile, turn order tile, carte personaggio/edificio/evento, piazzamento totem, risoluzione azioni, eventi, fine round e punteggio finale.

- TUI e GUI JavaFX. - Server unico con client Socket e RMI nello stesso match. - Disconnessione/riconnessione, sospensione, timeout, persistenza e leaderboard JDBC opzionale.

DIVISIONE DEL LAVORO

- Benedetto Oliva: model, carte, effetti, regole di dominio e test del model.
- Giorgio Ortona: Socket/RMI, client controller, riconnessioni/disconnessioni, GUI e integrazione.
- Alessandro Mazzeo: controller, stati di gioco, flussi di partita e regressioni.
- Camillo Nicoletti: TUI, comandi utente, persistenza, leaderboard e deliverable.

SCELTE ARCHITETTURALI E PATTERN

Il progetto segue MVC con server autoritativo: il client invia intenzioni, mentre il server valida le azioni e mantiene lo stato ufficiale. Il client conserva solo un ClientModel locale, aggiornato dai messaggi ricevuti.

Scelta / pattern	Dove viene usato	Motivo
State	LobbyState, TotemPlacementState, ActionResolutionState, SuspendedState, EndGameState	separare le regole valide in ogni fase
Command	GameAction e sottoclassi	rappresentare le azioni inviate dal client
DTO / Protocol messages	serverbound, clientbound	trasportare dati senza legare rete e logica
Factory + Strategy	ClientConnectionFactory, ClientSideConnection	ricreare Socket/RMI senza cambiare controller
Adapter	Socket*Connection, RMI*Connection, VirtualView	uniformare trasporti diversi e convertire eventi in messaggi
Dispatcher	MessageDispatcher, mappe handler nel client	evitare catene di instanceof sui messaggi
Observer	GameListener, callback setOnMessage, setOnDisconnect	notificare eventi senza dipendenze dirette
Visitor	carte ed effetti	specializzare il comportamento per tipo di carta/effetto
Snapshot	GameStartedMessage, GameSyncMessage	riallineare completamente un client
Supervisor / Retry	ClientConnectionSupervisor	tentare reconnect periodico ogni 2 secondi

Mesos - Mini report finale

PROTOCOLLO E PERSISTENZA

Il protocollo usa oggetti serializzabili. I ClientMessage includono LoginMessage, ReadyMessage, GameActionMessage, ReconnectMessage, DisconnectMessage, QuitMessage; i ServerMessage includono LoginAckMessage, GameStartedMessage, GameSyncMessage, StateChangedMessage, PlayerDisconnectedMessage, GameSuspendedMessage, GameResumedMessage.

Durante il gioco normale il server invia messaggi delta: solo cio che cambia. Il GameSnapshot completo viene costruito solo quando serve una sincronizzazione completa: inizio partita o riconnessione. La persistenza non salva lo snapshot, ma serializza il GameController in mesos-savestate.bin; al restart il server ricarica il controller e genera un nuovo snapshot per ogni client che si riconnette.

MESSAGGI PRINCIPALI

Direzione	Messaggi
Client -> Server	LoginMessage, ReadyMessage, GameActionMessage, ReconnectMessage, DisconnectMessage, QuitMessage
Server -> Client	LoginAckMessage, GameStartedMessage, GameSyncMessage, StateChangedMessage, PlayerDisconnectedMessage, GameSuspendedMessage, GameResumedMessage

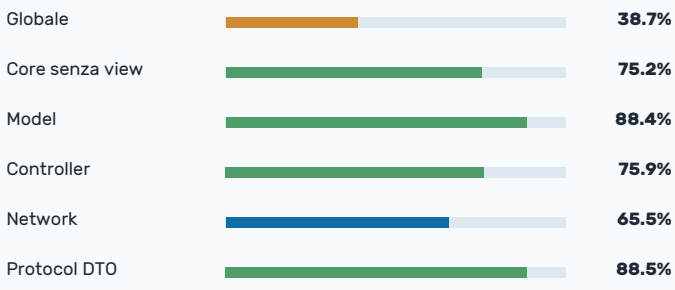
FLUSSO RECONNECT

1. **Guasto rilevato** — heartbeat, eccezione o onDisconnect
2. **Cleanup server** — LobbyManager pulisce la connessione
3. **Stato partita** — disconnessione e sospensione
4. **Retry client** — nuova richiesta ogni 2 sec
5. **Riallineamento** — ack, snapshot e stato corrente

TEST E COPERTURA

La suite contiene 95 classi di test su model, controller, stati, network, protocollo, persistenza e riconnessioni. La copertura e misurata con JaCoCo tramite mvn clean verify. GUI/TUI, launcher e JDBC leaderboard sono validati anche con prove manuali perche dipendono da interazione grafica, processo o database esterno.

Copertura istruzioni JaCoCo



Indicatori sintetici

Test class	95
Branch globale	36.0%
Linee globali	41.6%
Core linee	73.5%
Comando coverage	mvn clean verify
Validazione manuale	GUI/TUI, launcher, JDBC